Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ7 «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Разработка базы данных для поиска и организации досуга»

Студент ИУ7-64Б (Группа) _____ (Подпись, дата) М. В. Козин (И. О. Фамилия)

Руководитель курсовой работы _____ (Подпись, дата) Н. В. Назаренко (И. О. Фамилия)

2026 г.

РЕФЕРАТ

Расчетно-пояснительная записка 39 с., 10 рис., 22 табл., 1 прил., 22 источн.

досуг, мероприятие, база данных, postgresql, триггер.

Цель работы – разработка базы данных и приложения для поиска и организации досуга.

Выделены сущности предметной области, определены требования и роли пользователей. Разработана реляционная база данных для хранения мероприятий, участников и отзывов. Реализованы приложение JoinUp, разграничение доступа и триггер записи фактов удаления объявлений.

Проанализирована зависимость времени выполнения поиска от количества записей в таблице с индексацией и без нее. Для 10000 мероприятий среднее время составило 4,905 мс без индекса и 3,038 мс с ним.

СОДЕРЖАНИЕ

РЕФЕРАТ **2**

ВВЕДЕНИЕ **5**

1 Аналитическая часть **6**

1.1 Анализ предметной области 6

1.2 Анализ известных решений 6

1.3 Требования к системе 7

1.4 Формализация данных 8

1.5 Формализация ролей 10

1.6 Анализ моделей баз данных 12

1.7 Формализация задачи 12

2 Конструкторская часть **13**

2.1 Проектирование базы данных 13

2.2 Ограничения целостности 17

2.3 Нормализация базы данных 17

2.4 Ролевая модель 19

2.5 Триггер удаления мероприятия 19

2.6 Архитектура и взаимодействие с БД 20

3 Технологическая часть **22**

3.1 Выбор СУБД 22

3.2 Средства реализации 22

3.3 Реализация базы данных 23

3.4 Доступ к данным и запросы 24

3.5 Интерфейс и подготовка данных 25

3.6 Проверка работы 28

4 Исследовательская часть **30**

4.1 Цель и условия исследования 30

4.2 Методика измерения 31

4.3 Результаты исследования 31

ЗАКЛЮЧЕНИЕ **33**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **34**

ПРИЛОЖЕНИЕ А. Сценарий создания объектов базы данных 36

ВВЕДЕНИЕ

Для организации совместного досуга необходимо хранить сведения о мероприятиях, времени и месте проведения, участниках и отзывах. Размещение объявлений в общей ленте затрудняет поиск по заданным условиям и учет участников. Эти задачи решает специализированное приложение с базой данных.

Цель работы – разработка базы данных для поиска и организации досуга и приложения для взаимодействия с ней.

Для достижения цели необходимо:

1) проанализировать предметную область и известные решения;
2) определить требования, сущности и связи между ними;
3) спроектировать БД, ограничения целостности, роли и триггер сохранения истории удаления объявлений;
4) выбрать средства реализации и разработать приложение;
5) проверить корректность основных операций;
6) исследовать время поиска мероприятий с индексом по основным фильтрам и без него.

1 Аналитическая часть

1.1 Анализ предметной области

Поиск мероприятий для совместного досуга связан с выбором тематики, места и времени проведения. Объявления размещаются в разных группах и общих лентах, поэтому пользователю приходится просматривать множество публикаций, чтобы найти подходящее мероприятие. Если дата, адрес и условия участия указаны неполно или в разных сообщениях, сравнение вариантов занимает дополнительное время.

Для организации мероприятия нужно указать его тему, время и место проведения, опубликовать описание и учитывать участников. В общей ленте объявление быстро теряется среди других записей, а изменение сведений и учет откликов затруднены. Для упрощения этих задач необходима система, в которой можно быстро найти мероприятия по интересам, городу и дате, а также разместить свое объявление и вести учет участников.

1.2 Анализ известных решений

Рассмотрим следующие существующие решения для поиска и организации мероприятий:

1) Meetup – сервис публикации и поиска встреч по интересам с учетом участников [1—4];

2) Mobilizon – приложение для создания мероприятий, поиска и управления участием; организатор может разрешить запись без учетной записи [5—8];

3) Timepad – сервис публикации и поиска мероприятий, регистрации участников и продажи билетов [9—11];

4) группы в социальных сетях – публикация объявлений в ленте группы с обсуждением в комментариях.

Критерии сравнения и результаты представлены в таблицах 1.1 и 1.2. Для групп рассматриваются обычные публикации во ВКонтакте, без отдельных сервисов мероприятий. Универсальность означает возможность размещения мероприятий разных тематик.

Таблица 1.1 – Критерии сравнения известных решений

Критерий	**Описание**
Доступность в РФ	Возможность пользоваться приложением из России
Интеграция с соцсетями	Средства публикации, регистрации или общения через соцсети и мессенджеры
Универсальность	Размещение мероприятий разных тематик
Запись без аккаунта	Подтверждение участия без предварительного создания учетной записи
Фильтры поиска	Поиск мероприятий по месту и времени проведения

Таблица 1.2 – Результаты сравнения известных решений

Критерий	**Meetup**	**Mobilizon**	**Timepad**	**Группы соцсетей**	**JoinUp**
Доступность в РФ	Нет	Нет	Да	Да	Да
Интеграция с соцсетями	Да	Да	Да	Да	Да
Универсальность	Да	Да	Да	Да	Да
Запись без аккаунта	Нет	Да	Да	Нет	Нет
Фильтры поиска	Да	Да	Да	Нет	Да

При проверке из РФ сайты Meetup и Mobilizon были недоступны. Для Mobilizon результат относится к проверенному публичному серверу.

Meetup позволяет привязать внешний чат, Mobilizon – поделиться событием в соцсетях, Timepad – встроить форму регистрации. В JoinUp используется ссылка на Telegram-чат. При записи через афишу Timepad личный кабинет создается автоматически. В JoinUp участие требует входа; поиск дополняют отзывы и рейтинг организатора.

1.3 Требования к системе

Основные требования и их приоритеты представлены в таблице 1.3. Высокий приоритет имеют основные операции и сохранность данных, средний – дополнительные сведения о пользователях и мероприятиях.

Таблица 1.3 – Основные требования к системе

Требование	**Приоритет**
Регистрация, вход и разграничение прав гостя, пользователя и администратора	Высокий
Создание, просмотр, изменение и удаление мероприятий	Высокий
Поиск по названию, городу, датам и категориям; присоединение и отказ от участия	Высокий
Запись факта удаления объявления триггером БД	Высокий
Профили, изображения, отзывы и рейтинг организатора	Средний
Постоянное хранение, целостность данных, хеширование паролей и роли доступа к БД	Высокий

Первые пять требований являются функциональными, последнее – нефункциональным. Основные ограничения: уникальность логина, однократное участие и один отзыв автора о мероприятии; оценка от 1 до 5, текст до 2000 символов. Создание мероприятий в прошлом запрещено. Изменять объявление может автор либо администратор; прошедшие мероприятия не редактируются. Журнал и удаление изменяются в одной транзакции.

1.4 Формализация данных

Основные группы данных представлены в таблице 1.4. Каждая запись имеет идентификатор. Участие пользователей и принадлежность мероприятий к категориям образуют связи «многие ко многим».

Таблица 1.4 – Группы данных и сведения о них

Данные	**Сведения**
Пользователь	Имя, возраст, логин, хеш пароля, город, Telegram, роль, аватар, дата регистрации
Мероприятие	Название, описание, город, время, организатор, локация, чат, обложка, признак удаления
Участие	Пользователь, мероприятие, роль участника
Локация	Название, адрес, координаты
Категория	Название, вышестоящая категория
Изображение	Имя файла, содержимое
Отзыв	Автор, мероприятие, оценка, текст, дата
Журнал удаления	Мероприятие, время удаления

На рисунке 1.1 представлена исходная диаграмма предметной области. Реализованная БД дополнена отзывами и журналом удаления; показанные на схеме подписка и дата вступления не используются приложением.

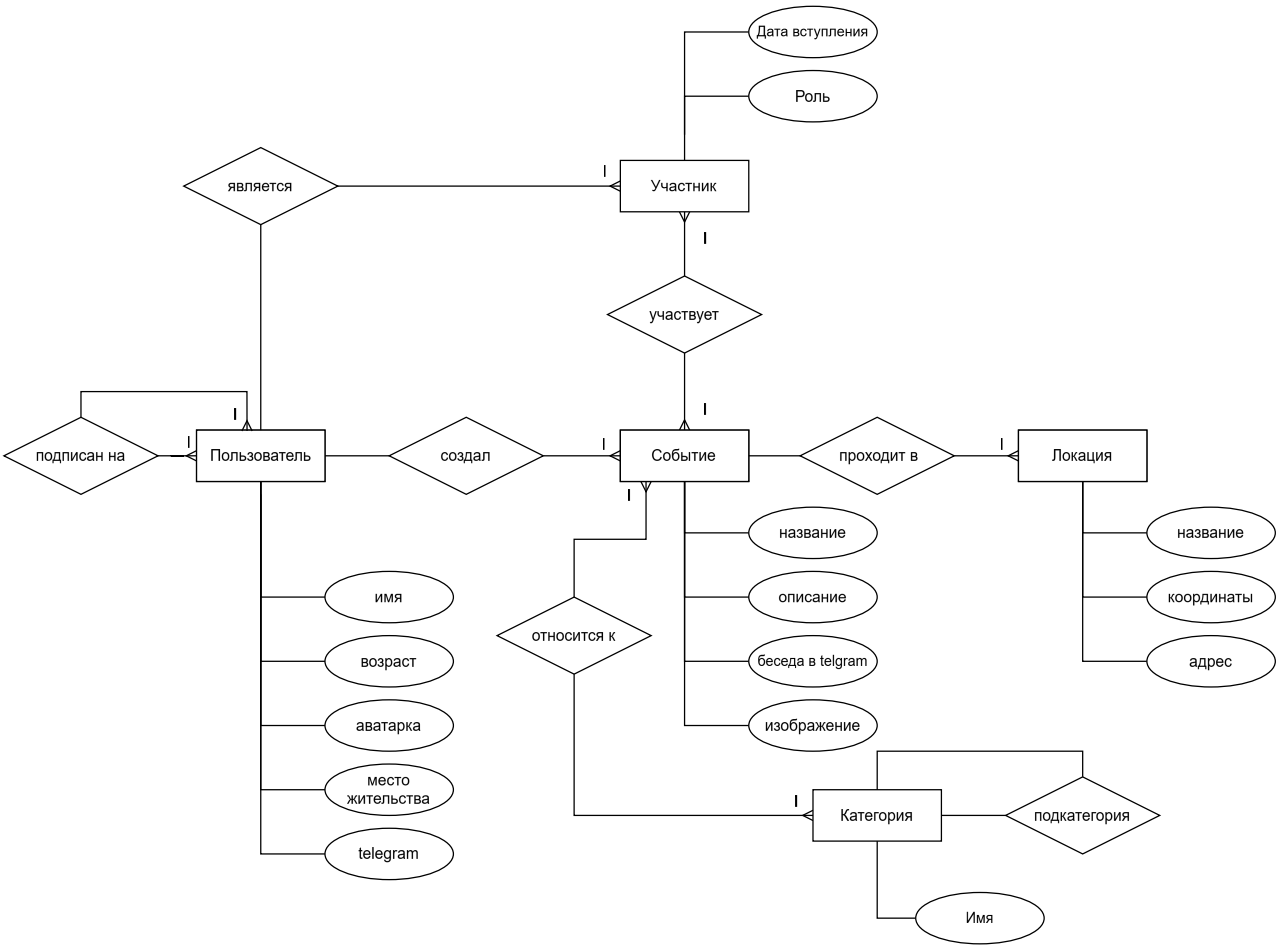


Рисунок 1.1 – Диаграмма сущность-связь

1.5 Формализация ролей

Роли приложения приведены в таблице 1.5, их действия – на рисунке 1.2.

Таблица 1.5 – Роли пользователей приложения

Роль	**Возможности**
Гость	Регистрация, вход, просмотр открытых изображений и отзывов
Пользователь	Поиск, создание и изменение своих мероприятий, участие, отзывы, работа с профилем
Администратор	Действия пользователя, изменение и удаление чужих мероприятий

Роли БД задают права чтения, изменения и администрирования данных. Сервер подключается с пользовательской ролью БД; права на объявления проверяет приложение.

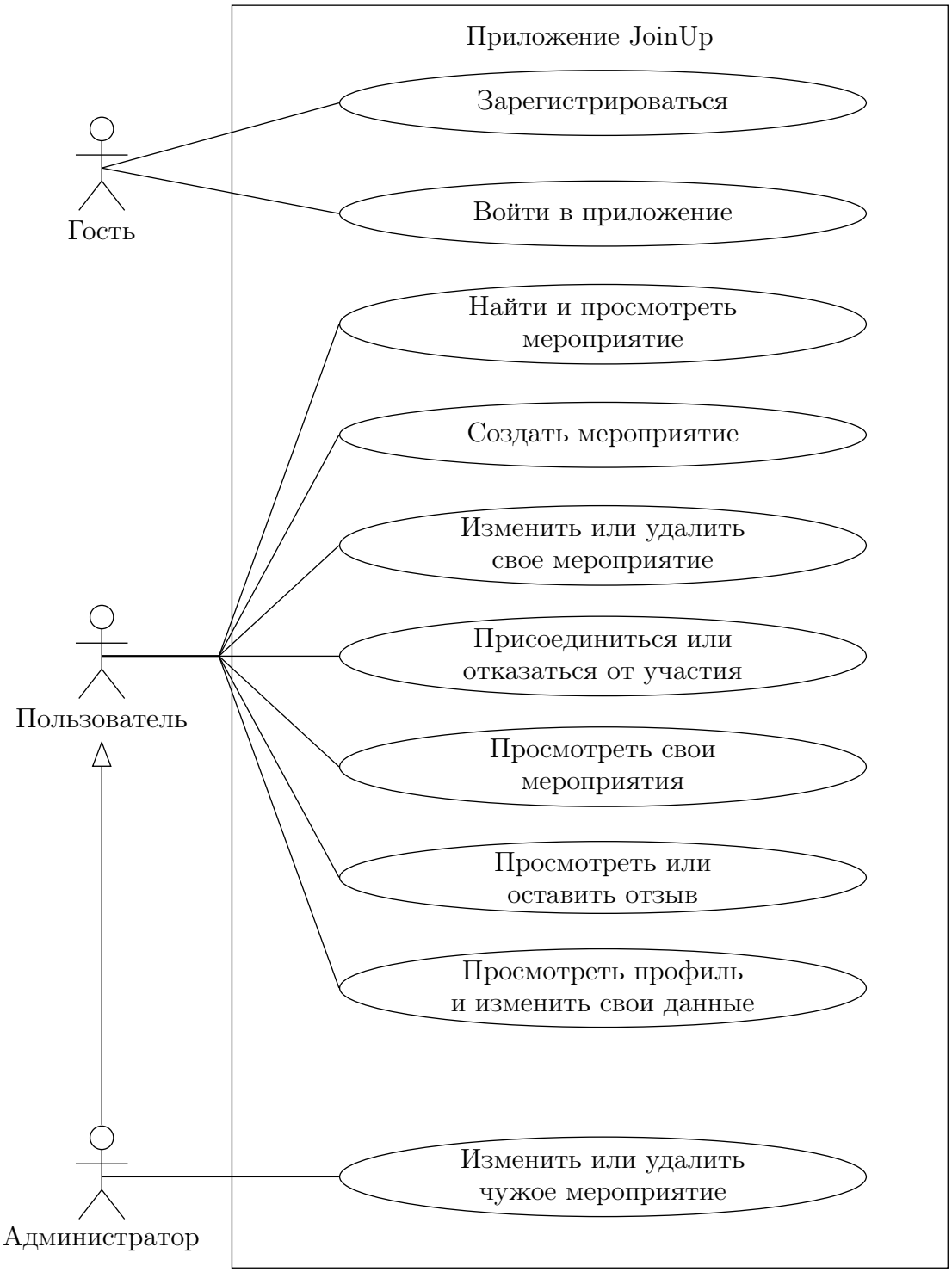


Рисунок 1.2 – Диаграмма прецедентов

1.6 Анализ моделей баз данных

Иерархическая модель представляет данные деревом: у подчиненной записи один родитель [12]. Она подходит для категорий, но связи пользователей с несколькими мероприятиями требуют дополнительных ссылок либо дублирования данных.

Документная модель хранит объекты с вложенными данными или ссылками [13]. Мероприятие можно хранить вместе с участниками, однако при копировании сведений о пользователе их изменение потребует обновления нескольких документов.

Реляционная модель представляет данные таблицами, связанными ключами [12]. Для JoinUp выбирается эта модель: состав атрибутов определен, участие и категории задаются отдельными отношениями, поиск выполняется отбором и соединением записей, рейтинг – агрегированием оценок.

1.7 Формализация задачи

Необходимо разработать реляционную БД и приложение для публикации и поиска мероприятий, учета участия и отзывов. Требуется обеспечить целостность данных, разграничение прав и запись фактов удаления объявлений триггером. После реализации необходимо проверить основные операции и исследовать зависимость среднего времени поиска от количества мероприятий с индексом по основным фильтрам и без него.

2 Конструкторская часть

2.1 Проектирование базы данных

На основе сущностей аналитической части определены таблицы пользователей, мероприятий, локаций, изображений, категорий, участников и отзывов. Связь мероприятий с категориями вынесена в отдельную таблицу. Факты удаления хранятся в журнале. В схеме также сохранены таблицы пресетов и подписок; приложение их не использует.

Пользователи, мероприятия, локации и изображения описаны в таблицах 2.1, 2.2 и 2.3. Указаны смысловые типы данных. Все атрибуты обязательны, кроме отмеченных как необязательные. Поле `id` – первичный ключ каждой таблицы, кроме связи мероприятий с категориями, где ключ составной.

Таблица 2.1 – Структура таблицы пользователей users

Атрибут	**Тип**	**Назначение и ограничения**
id	Идентификатор	Пользователь
name, city	Строка	Имя и город проживания
age	Целое число	Возраст, больше нуля
login	Строка	Уникальный логин
password	Строка	Хеш пароля
created_at	Дата и время	Регистрация
role	Строка	Роль приложения, по умолчанию user
telegram_login	Строка	Имя в Telegram, необязательное
avatar_id	Идентификатор	Ссылка на image, необязательная

Таблица 2.2 – Структура таблицы мероприятий event

Атрибут	**Тип**	**Назначение и ограничения**
id	Идентификатор	Мероприятие
name, city	Строка	Название и город проведения
description	Строка	Описание, необязательное
created_at, updated_at, event_date	Дата и время	Создание, последнее изменение и проведение
telegram_chat_url	Строка	Ссылка на чат, необязательная
creator_id	Идентификатор	Организатор, ссылка на users
location_id	Идентификатор	Место проведения, ссылка на location
image_id	Идентификатор	Обложка, ссылка на image, необязательная
deleted	Логический	Признак удаления, изначально ложный

Таблица 2.3 – Структура таблиц локаций и изображений

Таблица	**Атрибуты**	**Тип и назначение**
location	id	Идентификатор локации
location	name, address	Строки: название и адрес
location	longitude, latitude	Вещественные числа: долгота и широта
image	id	Идентификатор изображения
image	name	Строка: имя файла
image	data	Двоичные данные: содержимое файла

Категории, участие, отзывы, журнал и неиспользуемые таблицы описаны в таблицах 2.4, 2.5 и 2.6.

Таблица 2.4 – Структура таблиц категорий и их связи с мероприятиями

Таблица	**Атрибуты**	**Тип, назначение и ограничения**
category	id	Идентификатор категории
category	name	Строка: уникальное название
category	subcategory_id	Идентификатор: необязательная ссылка на category
event_category	event_id, category_id	Идентификаторы мероприятия и категории; внешние ключи и составной первичный ключ

Таблица 2.5 – Структура таблиц участия, отзывов и журнала

Таблица	**Атрибуты**	**Тип и назначение**
member	id	Идентификатор участия
member	user_id, event_id	Идентификаторы пользователя и мероприятия; внешние ключи
member	role	Строка: роль участника мероприятия
review	id	Идентификатор отзыва
review	event_id, author_id	Идентификаторы мероприятия и автора; внешние ключи
review	rating, text	Целое число: оценка; строка: текст отзыва
review	created_at	Дата и время создания
event_deletion_log	id	Идентификатор записи журнала
event_deletion_log	event_id	Идентификатор мероприятия; внешний ключ
event_deletion_log	deleted_at	Дата и время удаления

Таблица 2.6 – Сохраненные таблицы, не используемые приложением

Таблица	**Атрибуты**	**Тип и назначение**
preset	id, name	Идентификатор и строка: название пресета
preset	category_id, creator_id	Идентификаторы категории и автора; внешние ключи
subscribe	id	Идентификатор подписки
subscribe	subscriber_id, user_id	Идентификаторы подписчика и отслеживаемого пользователя; ссылки на users
subscribe	subscribe_at	Дата и время подписки

Диаграмма БД представлена на рисунке 2.1. Стрелки направлены от внешнего ключа к таблице, на которую он ссылается. Каждая связь имеет кратность «многие к одному»; со стороны дочерних записей допускается ноль элементов. Для необязательных внешних ключей связанная запись может отсутствовать. Обозначения PK и FK соответствуют первичному и внешнему ключам.

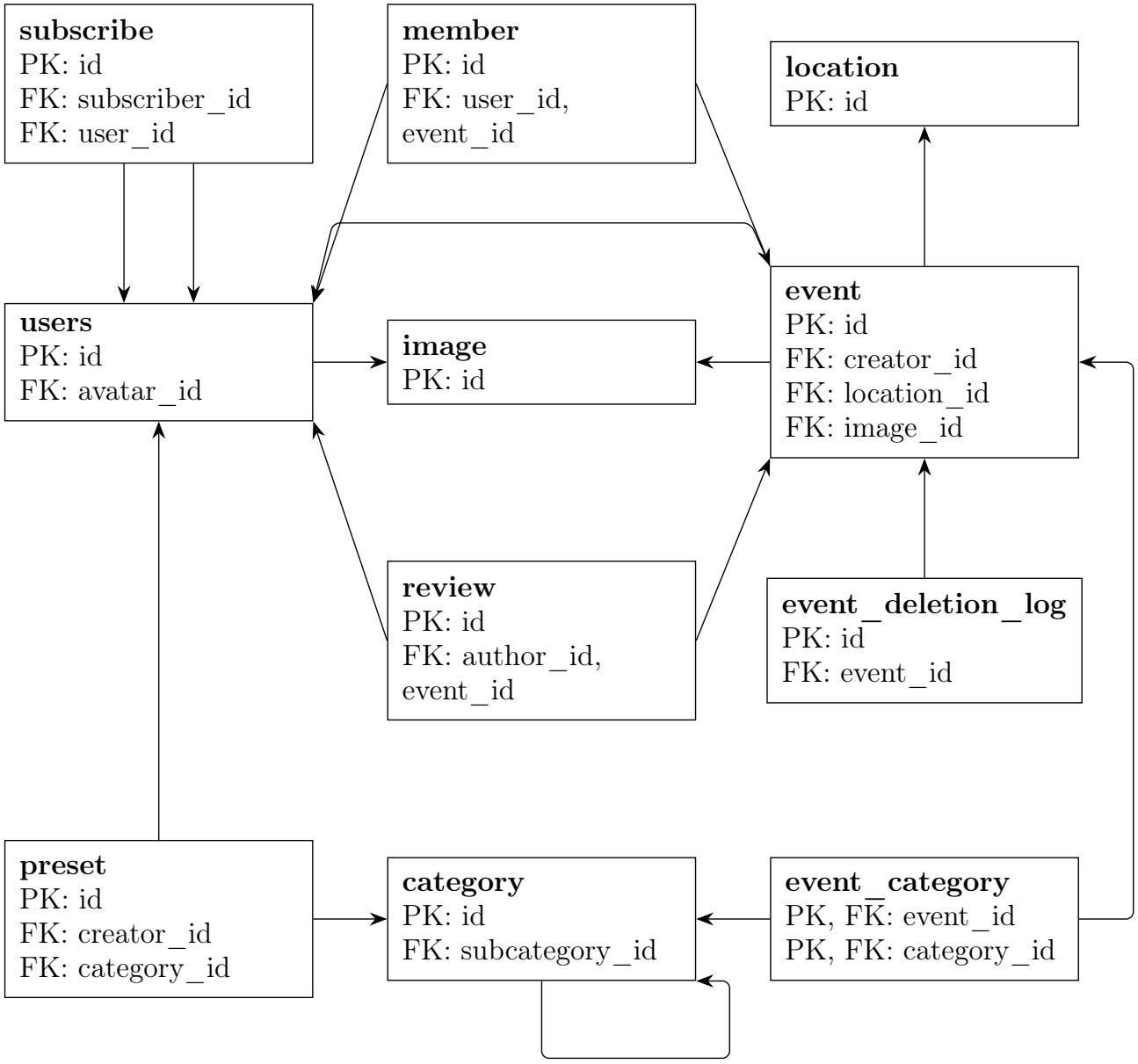


Рисунок 2.1 – Диаграмма базы данных

2.2 Ограничения целостности

Первичные ключи определяют записи, внешние ключи запрещают ссылки на отсутствующие объекты. Дополнительные ограничения приведены в таблице 2.7.

Таблица 2.7 – Ограничения БД и проверки приложения

Уровень	**Ограничения**
База данных	Уникальны логин, название категории, пары пользователь/мероприятие в member, мероприятие/автор в review и мероприятие/категория в event_category
База данных	Возраст больше нуля; оценка от 1 до 5; длина отзыва не более 2000 символов
Приложение	Создание и перенос мероприятия в прошлое запрещены; прошедшее мероприятие нельзя редактировать
Приложение	Изменение и удаление мероприятия доступны автору или администратору
Приложение	Отзыв разрешен после наступления времени неудаленного мероприятия, автор отзыва не является организатором; после удаления пробелов по краям текст содержит от 1 до 2000 символов

Удаление объявления изменяет его признак `deleted`; связанные записи сохраняются. При физическом удалении мероприятия каскадно удаляются участие, связи с категориями, отзывы и журнал. Физическое удаление пользователя каскадно удаляет его мероприятия, участия, отзывы, пресеты и обе стороны подписок. Удаление используемой локации, изображения или категории ограничивается внешними ключами. Эти правила не означают наличие соответствующих операций в интерфейсе.

2.3 Нормализация базы данных

Нормализация проверяется по функциональным зависимостям: одинаковые значения определяющих атрибутов должны задавать одинаковые значения зависимых атрибутов. Для третьей нормальной формы (3НФ) в каждой нетривиальной зависимости определяющий набор должен быть суперключом либо зависимый атрибут должен входить в потенциальный ключ [14].

Потенциальные ключи отношений перечислены в таблице 2.8. Каждый указанный ключ определяет все остальные атрибуты своей строки; для со-

ставных ключей определяющим является весь набор. Других зависимостей в принятой модели не задано.

Таблица 2.8 – Потенциальные ключи отношений

Отношение	**Потенциальные ключи**
users	id; login
category	id; name
member	id; (user_id, event_id)
review	id; (event_id, author_id)
event_category	(event_id, category_id)
event, location, image	id в каждом отношении
preset, subscribe, event_deletion_log	id в каждом отношении

Проверим нормальные формы:

1) Первая нормальная форма: атрибуты содержат одиночные значения выбранных доменов. Списки участников и категорий представлены отдельными строками связующих таблиц. Изображение хранится одним двоичным значением.

2) Вторая нормальная форма: в member роль зависит от пары пользователь/мероприятие, в review оценка, текст и время – от пары мероприятие/автор. Отдельный пользователь или мероприятие не определяет эти сведения. У event_category нет неключевых атрибутов; остальные потенциальные ключи состоят из одного атрибута.

3) Третья нормальная форма: сведения о пользователе, локации, категории и изображении не дублируются в связанных отношениях. Например, creator_id задает пользователя, но его имя не хранится в event; location_id задает локацию, но ее адрес вынесен в location. Между неключевыми атрибутами отношений дополнительных зависимостей нет. Все определяющие наборы из таблицы 2.8 являются ключами.

Таким образом, для заданных зависимостей отношения находятся в 3НФ. Названия локаций и их координаты не считаются уникальными; город мероприятия задается отдельно от адреса локации. Пара пользователей в subscribe не является ключом: ограничение ее уникальности отсутствует. Рейтинг организатора вычисляется по отзывам и отдельным атрибутом users не хранится.

2.4 Ролевая модель

Права подключений к БД распределены между ролями `joinup_guest`, `joinup_user` и `joinup_admin`. Матрица доступа приведена в таблице 2.9. В ней чтение, добавление, изменение и удаление обозначают соответствующие операции над строками.

Таблица 2.9 – Права ролей БД

Объекты	**Гость**	**Пользователь**	**Администратор**
event, location, image, category, event_category, review	Чтение	Чтение, добавление, изменение, удаление	Все права на таблицы
users, member, preset, subscribe, event_deletion_log	Нет	Чтение, добавление, изменение, удаление	Все права на таблицы
Последовательности идентификаторов десяти таблиц с полем id	Нет	Использование и чтение	Все права на последовательности
Подключение к БД, использование схемы public	Да	Да	Да

Полные права администратора относятся к указанным объектам, а не к управлению сервером БД. Сервер приложения всегда подключается как `joinup_user`; авторизация пользователя не меняет роль подключения. Поэтому принадлежность мероприятия автору и полномочия администратора приложения проверяются перед вызовом репозитория.

Пользовательская роль БД также имеет право добавления в журнал, необходимое триггерной функции. Журнал не защищен от прямого изменения этой ролью. Роль гостя БД имеет доступ к разрешенным таблицам целиком; фильтрация удаленных мероприятий выполняется приложением.

2.5 Триггер удаления мероприятия

Триггер `event_deletion_log_trigger` вызывает функцию `log_event_deletion` после обновления признака `deleted` для каждой измененной строки. Вход функции – прежнее и новое состояния строки;

результат – запись журнала при переходе признака из ложного значения в истинное.

Схема алгоритма представлена на рисунке 2.2. Повторное присваивание истинного значения не создает запись. Возврат признака в ложное значение также не записывается. При ошибке вставки исходное обновление откатывается вместе с действиями триггера. Журнал содержит идентификатор мероприятия и время удаления, без копии его названия и описания.

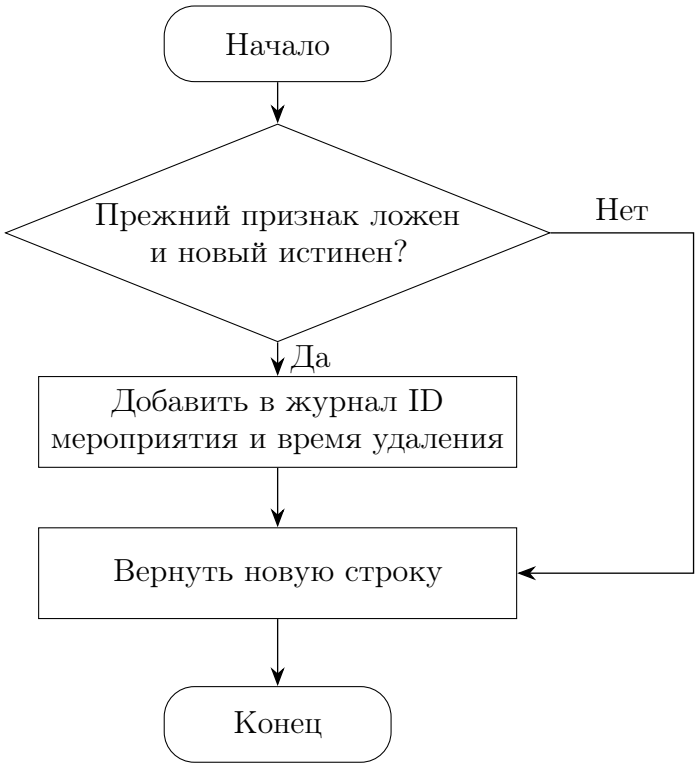


Рисунок 2.2 – Схема алгоритма функции записи факта удаления

2.6 Архитектура и взаимодействие с БД

Приложение состоит из клиентской части в браузере, сервера и БД. Схема представлена на рисунке 2.3. Контроллеры принимают запросы и формируют ответы, сервисы проверяют правила предметной области, репозитории выполняют запросы к БД. Для выбора и отображения места браузер обращается к Яндекс Картам. Ссылка на Telegram-чат хранится как атрибут мероприятия.

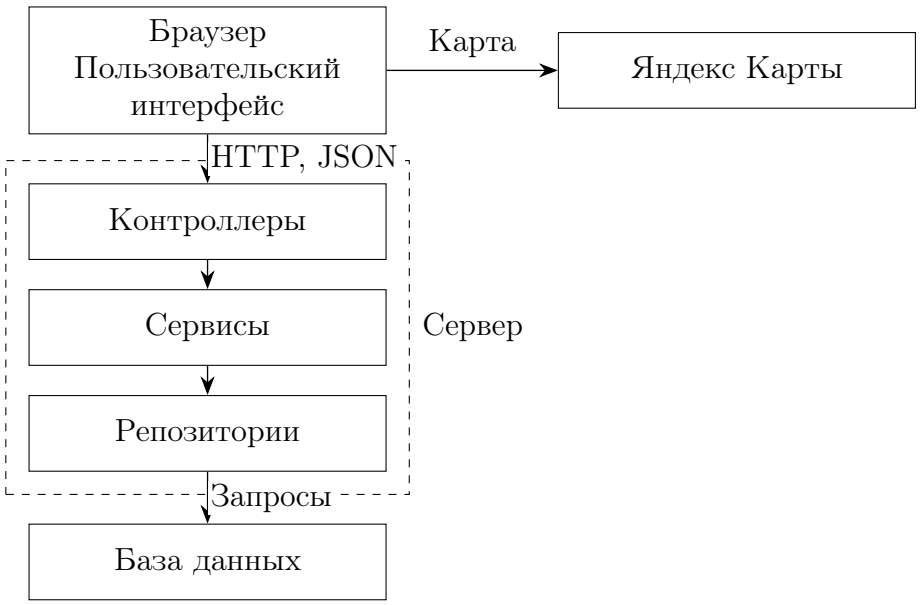


Рисунок 2.3 – Архитектура приложения

Сервер использует пул соединений и параметризованные запросы. Объект управления транзакцией передается репозиториям через контекст операции. Границы транзакций приведены в таблице 2.10.

Таблица 2.10 – Транзакции изменения данных

Операция	**Действия в одной транзакции**
Создание мероприятия	Проверка ссылок, добавление локации и мероприятия
Загрузка обложки	Добавление изображения и изменение ссылки на него в мероприятии
Создание отзыва	Чтение мероприятия, проверка условий и отсутствия отзыва, добавление отзыва
Мягкое удаление	Обновление признака удаления и вставка в журнал триггером в рамках одного оператора

Уникальные ограничения предотвращают повторное участие и повторный отзыв при одновременных запросах. Обновление локации и мероприятия выполняется без общей транзакции; проверки времени и прав не блокируют строку.

Вывод

Определены структура 11 таблиц, ограничения и права ролей; для принятых зависимостей показано соответствие 3НФ. Разработаны схема триггера, архитектура приложения и границы транзакций.

3 Технологическая часть

3.1 Выбор СУБД

В аналитической части выбрана реляционная модель. Для реализации рассмотрены PostgreSQL, MySQL и SQLite. Сравнение по требованиям проекта приведено в таблице 3.1. PostgreSQL предоставляет ограничения связей, транзакции, триггеры и права на объекты БД [15; 16]. MySQL также поддерживает перечисленные механизмы [17; 18]. SQLite поддерживает триггеры и транзакции [19; 20]; внешние ключи включаются отдельно для каждого соединения, а разграничение доступа командами `GRANT` и `REVOKE` отсутствует [21; 22].

Таблица 3.1 – Сравнение СУБД по требованиям проекта

Критерий	**PostgreSQL**	**MySQL**	**SQLite**
Внешние ключи	Да	Да	Да, после включения
Транзакции	Да	Да	Да
Триггеры для строк	Да	Да	Да
Роли и права на таблицы	Да	Да	Нет

Для JoinUp выбрана PostgreSQL 16: она позволяет реализовать таблицы, триггер и три роли подключений средствами одной СУБД. MySQL также отвечает этим требованиям. Выбор PostgreSQL согласуется с примененным в проекте драйвером `pgx` и существующими SQL-миграциями.

3.2 Средства реализации

Сервер написан на Go 1.25. Для маршрутизации HTTP-запросов используется Echo 5, для доступа к PostgreSQL – `pgx/v5`, для применения SQL-миграций – Goose 3. Клиентская часть написана на React 18 и TypeScript; сборку выполняет Vite. Версии взяты из `app/go.mod` и `frontend/package.json`.

HTTP-сервер принимает запросы по маршрутам `/api/v1`, проверяет данные и вызывает сервисы. Сервисы проверяют правила приложения, а репозитории выполняют SQL-запросы. Схема взаимодействия показана на рисунке 2.3. Доступ к операциям с мероприятиями и профилем определяется токеном пользователя; подключение сервера к БД выполняется под ролью

joinup_user. Роли приложения и роли подключений не совпадают.

3.3 Реализация базы данных

Миграции выполняются Goose в порядке имен файлов: создание основных таблиц, категорий и уникальности участия, отзывов, журнала удаления, индекса поиска и прав ролей. Команда make migrate_up применяет их через app/cmd/migrations. Структура всех 11 таблиц и связи уже приведены в конструкторской части; здесь рассматриваются конкретные средства PostgreSQL.

Идентификаторы основных таблиц созданы типом serial; у review и event_deletion_log используется bigserial. Для названий и описаний применен text, для времени – timestamp, для признака удаления – boolean, для изображения – bytea, для координат – float. Внешние ключи связывают мероприятие с автором и локацией, отзыв – с мероприятием и автором. Ограничение UNIQUE установлено для пары мероприятие/автор отзыва; уникальный индекс – для пары пользователь/мероприятие в member. Оценка ограничена значениями от 1 до 5, длина текста отзыва – 2000 символами. Проверка непустого текста после удаления пробелов выполняется в сервисе.

Для поиска создан частичный индекс event_search_city_date_idx по полям city, event_date, id при deleted = false. Он соответствует фильтрации по городу и диапазону времени; исследование его влияния приведено в исследовательской части.

Функция log_event_deletion вызывается построчным триггером после обновления поля deleted в event. Существенный фрагмент реализации:

```
if old.deleted = false and new.deleted = true then
    insert into event_deletion_log (event_id) values (new.id);
end if;
```

Алгоритм показан на рисунке 2.2. При повторной установке признака удаления запись в журнал не добавляется. Вставка в журнал и исходное обновление выполняются атомарно в рамках одного SQL-оператора.

Миграция ролей создает joinup_guest, joinup_user и joinup_admin, затем выдает права на схему, таблицы и последовательности. Например, гостю разрешено чтение мероприятий, локаций, изображений, категорий и отзывов:

```
grant select on event, location, image, category,
```

```
event_category, review to joinup_guest;
```

Полная матрица прав представлена в таблице 2.9. Роль `joinup_user` имеет право записи в журнал, которое требуется функции триггера; она может изменять журнал и напрямую. При авторизации в приложении роль подключения не переключается. SQL-сценарий создания объектов БД приведен в приложении А.

3.4 Доступ к данным и запросы

Репозитории используют пул соединений `pgxpool.Pool`. Значения фильтров и изменяемых полей передаются параметрами `$1`, `$2` и т. д. Объект управления транзакцией передается через контекст операции; репозиторий выбирает транзакцию из контекста или пул соединений. Создание мероприятия, загрузка его обложки и создание отзыва используют явные транзакции, перечисленные в таблице 2.10.

Поиск выполняется методом `SearchEvents`. Запрос исключает удаленные объявления, соединяет `event` с `location`, применяет фильтры названия, дат и города. Для выбора мероприятий, содержащих все указанные категории, применяется подзапрос с группировкой по мероприятию и условием `HAVING COUNT(DISTINCT category_id)`. Результаты упорядочиваются по времени и идентификатору. После основного запроса сведения об авторах, участниках и категориях загружаются отдельными запросами.

Соответствие запросов приложения основным видам SQL приведено в таблице 3.2.

Таблица 3.2 – Виды запросов в репозиториях PostgreSQL

Вид запроса	**Пример применения**
Простой	Чтение пользователя по логину из `users`
Многотабличный	Поиск мероприятий с присоединением `location`
Агрегатный	Средняя оценка организатора через `AVG`
С группировкой	Подбор мероприятий по всем выбранным категориям
С подзапросом	Подсчет отзывов о мероприятиях организатора в `GetUser`
Изменение данных	Добавление мероприятия и обновление признака удаления

Рейтинг организатора не хранится отдельным полем: метод `GetUser` получает его из отзывов агрегатной функцией `AVG`.

3.5 Интерфейс и подготовка данных

В клиентском приложении имеются экраны регистрации, входа, поиска, созданных мероприятий, участий, профиля и карточки мероприятия. Поиск с заданными фильтрами показан на рисунке 3.1. В карточке мероприятия (рисунок 3.2) видны описание, место проведения, карта и ссылка на чат. Пользователь может присоединиться к мероприятию, автор – изменить или удалить его. Изображения загружаются через сервер; для выбора адреса и отображения карты используются API Яндекса при наличии настроенных ключей.

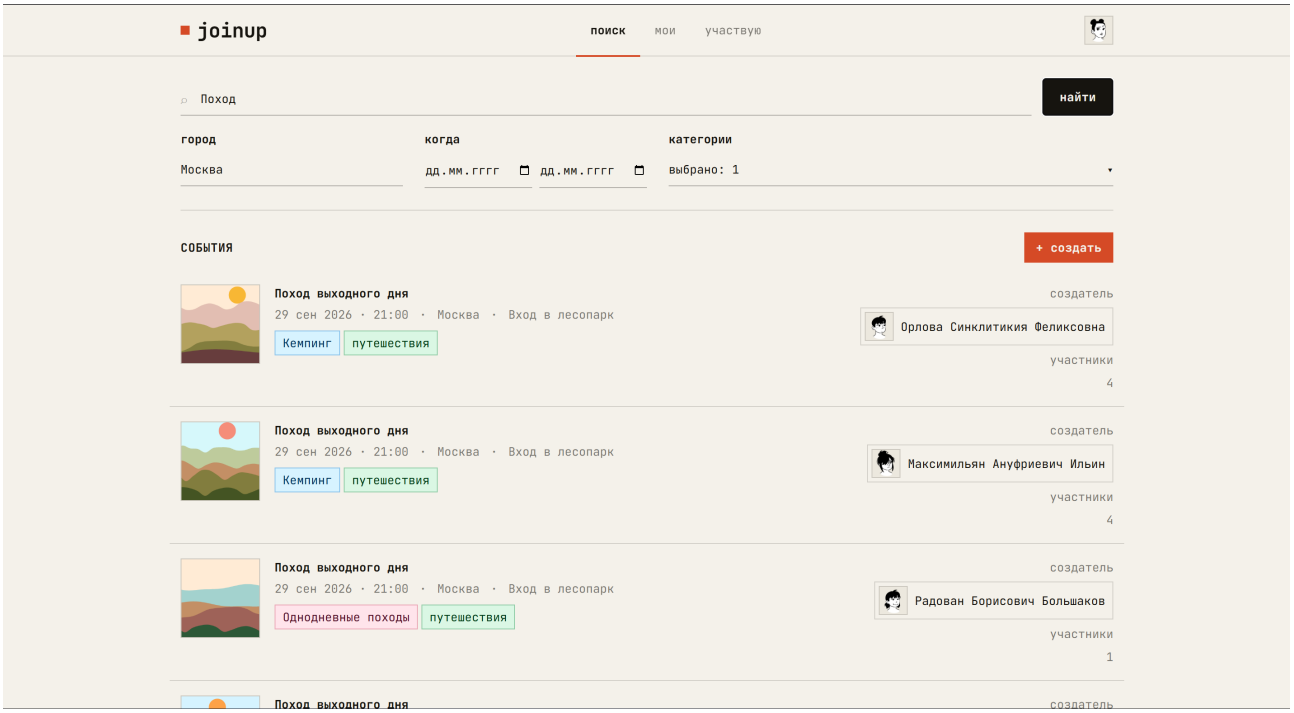


Рисунок 3.1 – Поиск мероприятий с фильтрами

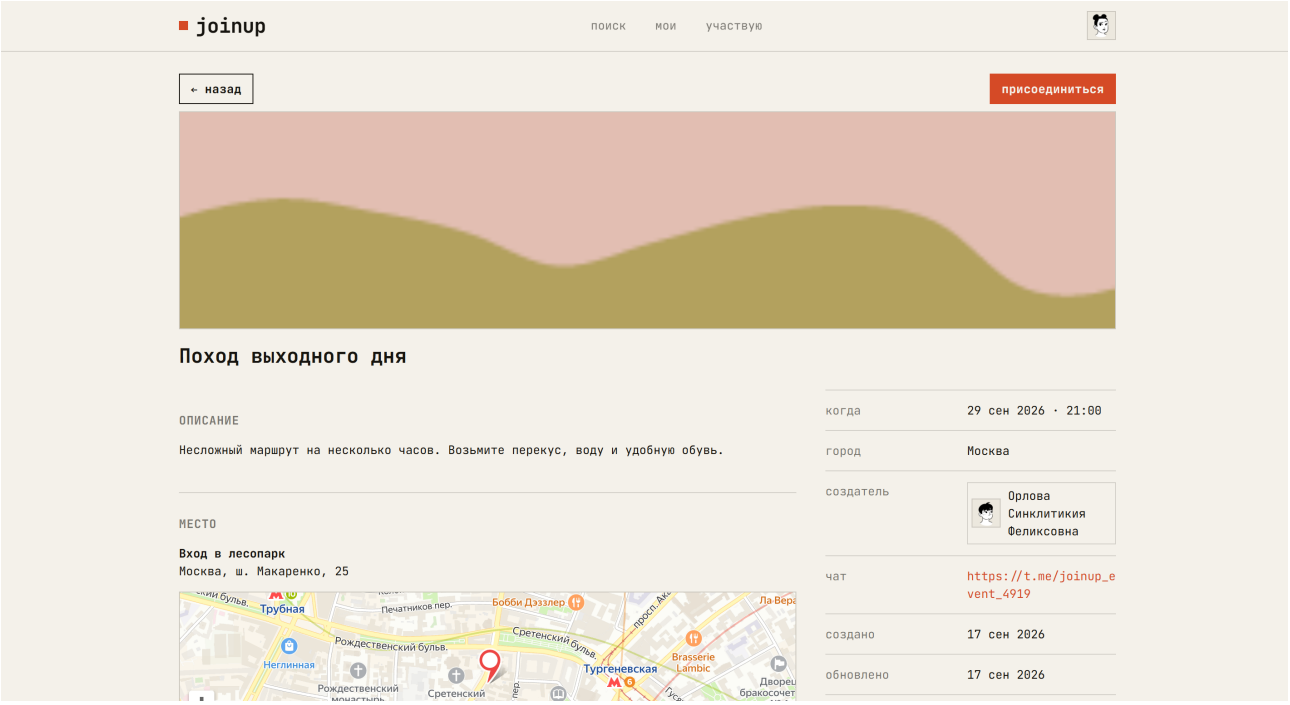


Рисунок 3.2 – Карточка мероприятия

Режим редактирования мероприятия представлен на рисунке 3.3, профиль пользователя с показателями участия и рейтингом – на рисунке 3.4.

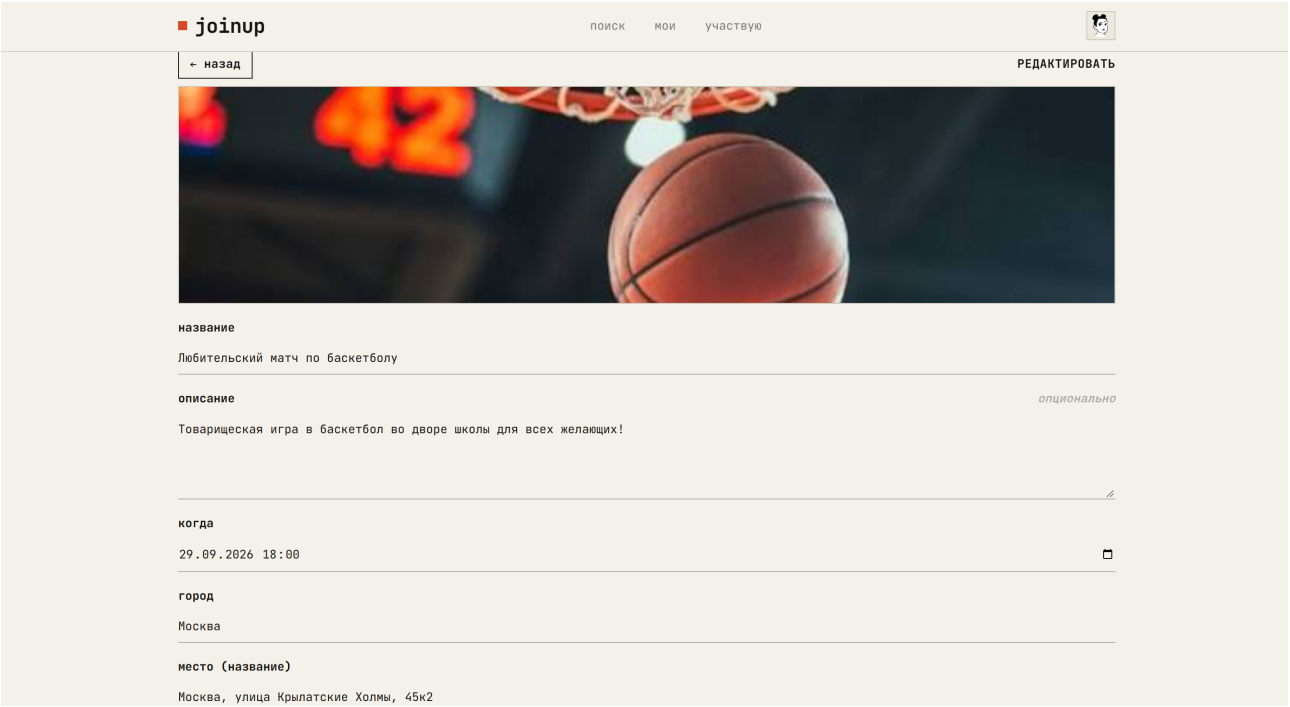


Рисунок 3.3 – Редактирование мероприятия

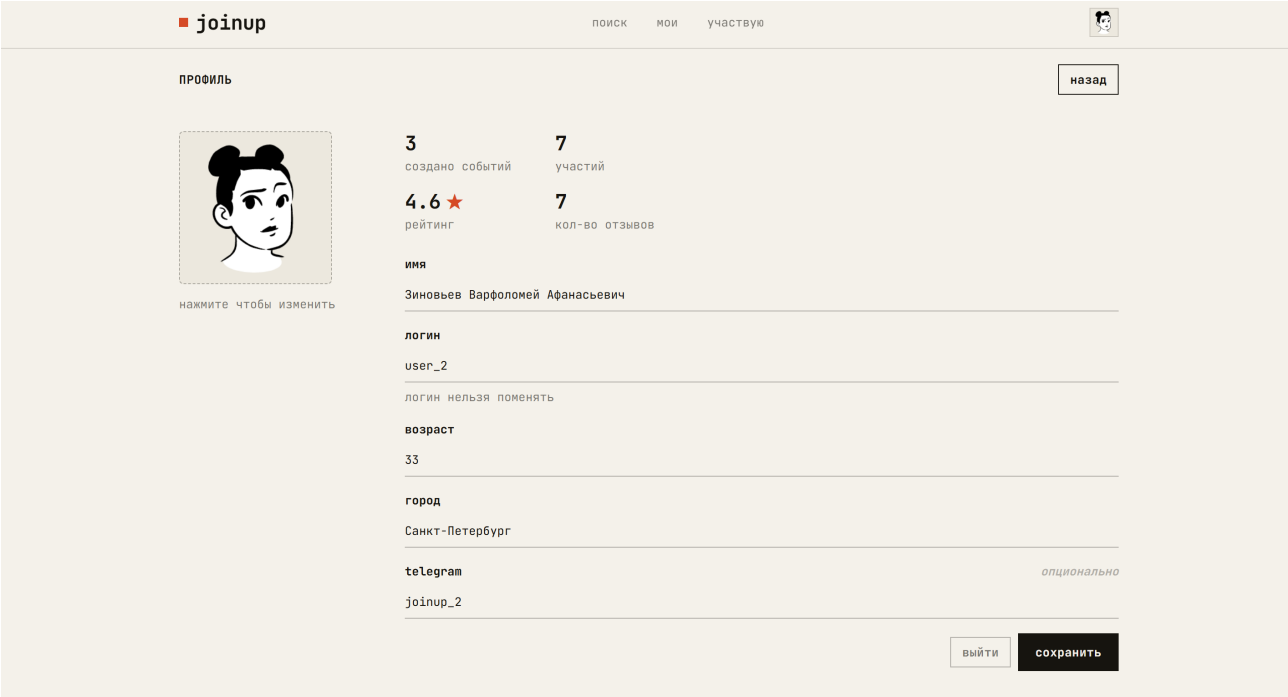


Рисунок 3.4 – Профиль пользователя

Учебные данные формирует `app/scripts/seed.py`. Он использует Faker с локалью `ru_RU`, справочные значения из `seed_data.json` и Pillow для генерации PNG. Сценарий создает 5001 пользователя; каждый создает от 1 до 3 мероприятий, участвует в нескольких мероприятиях, часть пользователей оставляет отзывы. После заполнения примерно 5% мероприятий помечаются удаленными, что вызывает триггер журнала. Затем учебные аватары и обложки были заменены изображениями DiceBear. Количества строк на 28.09.2026 приведены в таблице 3.3. Справочник содержит 11 направлений и по 8 конкретных тем в каждом.

Таблица 3.3 – Количество строк в таблицах БД

Таблица	**Строк**	**Таблица**	**Строк**
users	5001	event	10113
location	10113	image	15114
member	32406	category	99
event_category	20224	review	12853
preset	5001	subscribe	17589
event_deletion_log	505		

3.6 Проверка работы

В проекте имеются модульные проверки сервисов пользователя и мероприятия и интеграционные HTTP-проверки с PostgreSQL. Модульные проверки выполнены командой `go test ./internal/service`; команда завершилась успешно. Интеграционные проверки не запускались.

При ранее проведенной проверке HTTP-сервера получены результаты из таблицы 3.4. Для поиска заданы название «Разговорный клуб английского» и город Москва. После проверки учебные данные изменялись; значения ответов в таблице относятся к ее исходному состоянию.

Таблица 3.4 – Проверка запросов к приложению

Сценарий	**Ожидалось**	**Получено**
Вход `user_2` с верным паролем	HTTP 200	HTTP 200
Вход `user_2` с неверным паролем	HTTP 401	HTTP 401
Поиск без токена	HTTP 401	HTTP 401
Поиск по названию и городу	Неудаленные мероприятия	HTTP 200; 175 записей, удаленных нет
Карточка мероприятия с `id=1`	Данные мероприятия	HTTP 200; 3 участника, 2 категории
Профиль пользователя с `id=2`	Данные пользователя	HTTP 200; 3 созданных мероприятия, 5 участий
Отзывы мероприятия с `id=1`	Список отзывов	HTTP 200; 3 отзыва

Сохраненный протокол `app/scripts/permissions.csv` содержит результаты шести запросов с тремя учетными именами БД. Во всех шести строках фактический результат совпадает с ожидаемым. Сводка приведена в таблице 3.5.

Таблица 3.5 – Результаты сохраненной проверки прав БД

Роль	**Проверенные операции**	**Итог**
`joinup_guest`	Чтение `event`; запрет изменения `event` и чтения `users`	3 из 3
`joinup_user`	Чтение и изменение `event`	2 из 2
`joinup_admin`	Удаление из `event`	1 из 1

Изменяющие запросы этого протокола не затрагивали строки: применялось условие `WHERE false`. Поэтому протокол подтверждает доступ к

указанным операциям, но не проверяет все объекты матрицы прав и действие триггера.

Для функции `log_event_deletion` выделены классы эквивалентности в таблице 3.6. Проверка проведена на мероприятии с `id=1` внутри транзакции с последующим откатом. Указано изменение числа строк журнала для этого мероприятия.

Таблица 3.6 – Классы входных данных триггерной функции

Изменение признака	**Ожидалось**	**Получено**
`false -> true`	+1	+1
`false -> false`	0	0
`true -> true`	0	0
`true -> false`	0	0

Изменение одного только названия мероприятия также не увеличило число записей журнала. После отката транзакции признак удаления и число записей вернулись к исходным значениям. Сценарии записи через HTTP (создание мероприятия, участие, добавление отзыва) в этой проверке не охвачены.

Вывод

Реализованы база данных PostgreSQL, сервер Go и клиент React. Миграции создают таблицы, ограничения, индекс, триггер и роли; приложение обращается к БД через пул соединений и репозитории. Модульные проверки прошли, сохраненный протокол содержит шесть совпадений по правам. Проверены ответы сервера и четыре класса переходов признака удаления; сценарии записи через интерфейс требуют отдельной проверки.

4 Исследовательская часть

4.1 Цель и условия исследования

Цель исследования – определить, как меняется время запроса поиска при наличии индекса по городу и дате и без него для 1000, 2000, 5000 и 10000 мероприятий. Использован запрос метода `SearchEvents`, который соединяет мероприятие с локацией, исключает удаленные объявления, применяет фильтры и сортирует результат по дате и идентификатору. Загрузка категорий, участников и авторов после основного запроса не измерялась.

В обоих вариантах заданы город Москва и интервал от 26.08.2026 до 25.10.2026; фильтры названия и категорий не заданы. Опорная дата набора данных – 25.09.2026. Четыре объема являются вложенными частями одного сгенерированного набора. Число локаций постоянно и равно 12006; около 5% мероприятий помечены удаленными.

Исследуемый индекс имеет те же поля и условие, что и индекс приложения `event_search_city_date_idx`:

```
create index research_city_date_idx
    on event (city, event_date, id)
    where deleted = false;
```

Поле `city` отвечает условию равенства, `event_date` – диапазону дат, а `id` входит в порядок сортировки. Вариант без исследуемого индекса сохраняет первичный ключ временной таблицы; сравнивается действие именно этого дополнительного индекса.

Измерения проводились на компьютере с процессором AMD Ryzen 7 7700 и 32 ГБ оперативной памяти под управлением Windows 11. PostgreSQL 16.11 работала в WSL2; для выполнения запросов использовалась библиотека `psycopg` 3.3.6. Данные размещены во временных таблицах. Параллельное выполнение и JIT отключены; планировщику заданы `shared_buffers` 128 МиБ, `work_mem` 4 МиБ и `random_page_cost=4`. Кеш перед измерениями прогревался и не сбрасывался. В сохраненном протоколе указаны 12 логических процессоров, доступных среде WSL2.

4.2 Методика измерения

Для каждой комбинации объема данных и наличия индекса проведено 10 блоков по 50 измерений. Перед каждым блоком выполнено 10 прогревочных запросов. Порядок вариантов с индексом и без него чередовался между блоками. Таким образом, на каждую точку графика приходится 500 измерений, всего получено 4000 измерений. Все значения включены в расчет среднего.

Измеряемая величина – серверное время `Execution Time` команды `EXPLAIN (ANALYZE, BUFFERS, TIMING OFF, FORMAT JSON)`. Время планирования записано отдельно и в результат не входит. Среднее вычислено по 500 значениям. Погрешность на графике и в таблице рассчитана как $2{,}262s/\sqrt{10}$, где s – выборочное стандартное отклонение десяти средних значений по блокам. Это оценка 95%-го доверительного интервала при независимости блоков; из-за возможной корреляции блоков ее следует использовать только для ориентировочного сравнения.

4.3 Результаты исследования

Средние значения из 4000 исходных строк протокола приведены в таблице 4.1. Число найденных строк при каждом объеме одинаково для обоих вариантов.

Таблица 4.1 – Время выполнения запроса поиска

Записей	**Без индекса, мс**	**С индексом, мс**	**Найдено**
1000	$0{,}325 \pm 0{,}005$	$0{,}199 \pm 0{,}004$	57
2000	$1{,}824 \pm 0{,}054$	$1{,}583 \pm 0{,}077$	105
5000	$2{,}673 \pm 0{,}080$	$1{,}973 \pm 0{,}043$	287
10000	$4{,}905 \pm 0{,}320$	$3{,}038 \pm 0{,}203$	600

Зависимость времени от числа мероприятий представлена на рисунке 4.1. Сплошная линия с кругами соответствует варианту без индекса, пунктирная с квадратами – варианту с индексом; обозначения различимы при черно-белой печати.

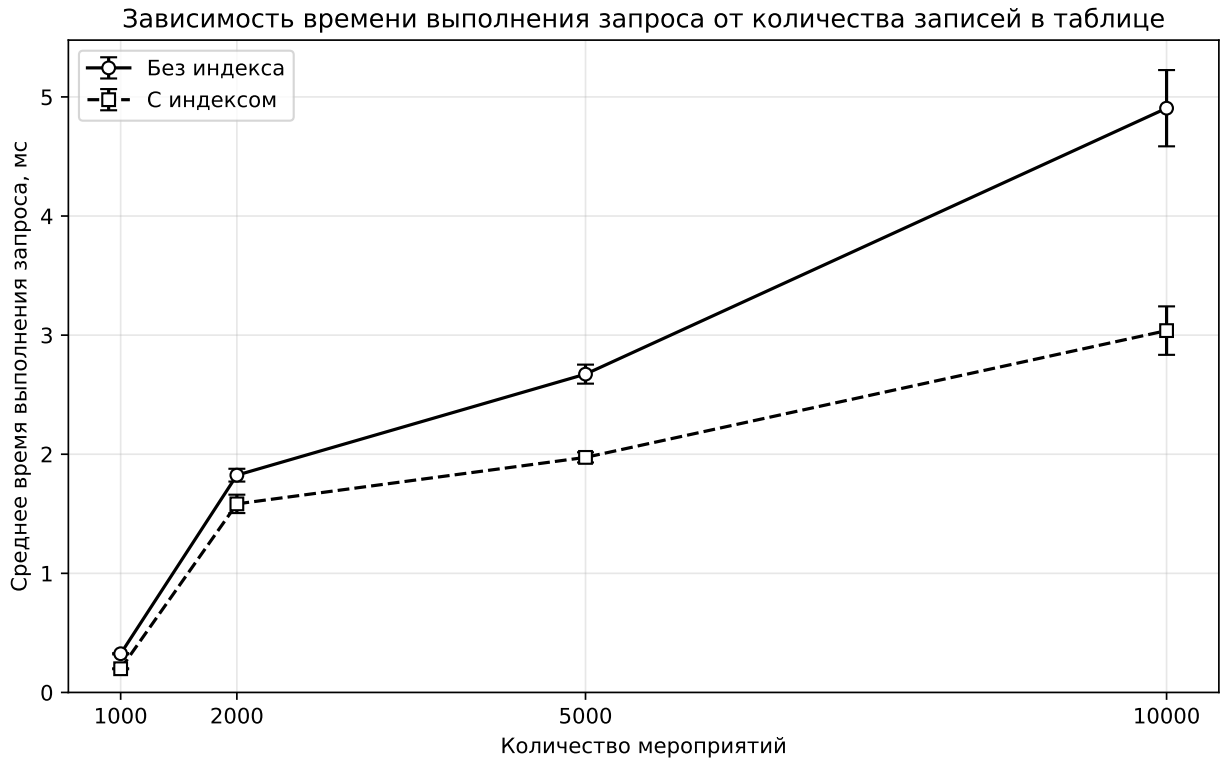


Рисунок 4.1 – Зависимость времени поиска от числа мероприятий

В сохраненных планах запрос без дополнительного индекса сканирует таблицу `event` последовательно. При наличии индекса применяется `Bitmap Index Scan` с последующим `Bitmap Heap Scan`. Для 1000 мероприятий соединение с `location` выполнено вложенным циклом, для остальных объемов – хеш-соединением. Смена способа соединения может влиять на рост времени между 1000 и 2000 строками; считать ее единственной причиной нельзя.

Вывод

Во всех четырех наборах среднее время с индексом меньше, чем без него. Для 10000 мероприятий оно составило 3,038 мс вместо 4,905 мс, то есть сократилось примерно на 38%. Результат относится к одному SQL-запросу, заданным фильтрам, синтетическим данным и прогретому кешу временных таблиц. Он не характеризует время загрузки страницы или работу всего приложения под нагрузкой.

ЗАКЛЮЧЕНИЕ

В работе проанализированы предметная область и известные решения, определены требования к системе. Построена модель данных, спроектированы таблицы и связи, рассмотрены ограничения целостности и соответствие третьей нормальной форме.

Разработаны база данных PostgreSQL и приложение JoinUp для поиска и организации мероприятий. В БД настроены роли и права доступа, индекс поиска и триггер, который записывает факт перехода мероприятия в состояние удаленного. Проверены отдельные запросы приложения, права ролей БД и переходы признака удаления. Сценарии записи через HTTP в проведенную проверку не входили.

По пяти критериям сравнения JoinUp отвечает четырем, а Timepad – пяти: в JoinUp не предусмотрена запись без учетной записи. Для учебной задачи реализованы поиск, публикация и участие в мероприятиях.

Исследование основного запроса поиска показало уменьшение среднего времени выполнения при наличии индекса для четырех объемов данных от 1000 до 10000 мероприятий. При 10000 мероприятий получены 4,905 мс без индекса и 3,038 мс с ним. Полученный результат относится к условиям проведенного исследования.

Результаты могут использоваться в учебных проектах с реляционной БД. Дальнейшая работа – проверка сценариев записи через интерфейс.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Creating an event – Meetup [Электронный ресурс]. — URL: https://help.meetup.com/hc/en-us/articles/39790436736525-Creating-an-event (дата обращения: 24.09.2026).
2. Finding an event – Meetup [Электронный ресурс]. — URL: https://help.meetup.com/hc/en-us/articles/39235072484109-Finding-an-event (дата обращения: 24.09.2026).
3. How do I sign up for a Meetup account? – Meetup [Электронный ресурс]. — URL: https://help.meetup.com/hc/en-us/articles/39232700471181-How-do-I-sign-up-for-a-Meetup-account (дата обращения: 25.09.2026).
4. Setting up event chats and third-party messaging on Meetup – Meetup [Электронный ресурс]. — URL: https://help.meetup.com/hc/en-us/articles/48377081479693-Setting-up-event-chats-and-third-party-messaging-on-Meetup (дата обращения: 25.09.2026).
5. Create an event – Mobilizon [Электронный ресурс]. — URL: https://docs.mobilizon.org/1.%20Use/Events%20and%20Activities/1.create-events/ (дата обращения: 25.09.2026).
6. How to search with Mobilizon – Mobilizon [Электронный ресурс]. — URL: https://docs.mobilizon.org/1.%20Use/users/search/ (дата обращения: 25.09.2026).
7. About Mobilizon – Mobilizon [Электронный ресурс]. — URL: https://docs.mobilizon.org/about/ (дата обращения: 25.09.2026).
8. Share, report, edit an event – Mobilizon [Электронный ресурс]. — URL: https://docs.mobilizon.org/1.%20Use/Events%20and%20Activities/3.event-action/ (дата обращения: 25.09.2026).
9. Как создать страницу мероприятия на Timepad – Timepad [Электронный ресурс]. — URL: https://help.timepad.ru/ru_RU/start/105486-timepad (дата обращения: 25.09.2026).
10. *Киселев А.* Как я могу узнавать о интересных событиях в моем городе – Timepad [Электронный ресурс]. — URL: https://help.timepad.ru/ru_RU/149382/80135 (дата обращения: 25.09.2026).
11. Возможности Timepad – Timepad [Электронный ресурс]. — URL: https://timepad.ru/about/product/ (дата обращения: 25.09.2026).
12. What is data modeling? – IBM [Электронный ресурс]. — URL: https://www.ibm.com/think/topics/data-modeling (дата обращения: 24.09.2026).

13. Data Modeling in MongoDB – Database Manual [Электронный ресурс]. — URL: https://www.mongodb.com/docs/manual/data-modeling/ (дата обращения: 24.09.2026).

14. Normalization: From Anomalies to Normal Forms – Aalto OpenCS [Электронный ресурс]. — URL: https://opencs.aalto.fi/en/courses/databases/part-8/4-normalization (дата обращения: 25.09.2026).

15. Supported Features – PostgreSQL 16 Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/16/features-sql-standard.html (дата обращения: 25.09.2026).

16. Privileges – PostgreSQL 16 Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/16/ddl-priv.html (дата обращения: 25.09.2026).

17. Privileges Provided by MySQL – MySQL 8.4 Reference Manual [Электронный ресурс]. — URL: https://dev.mysql.com/doc/refman/8.4/en/privileges-provided.html (дата обращения: 25.09.2026).

18. InnoDB Transaction Model – MySQL 8.4 Reference Manual [Электронный ресурс]. — URL: https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-model.html (дата обращения: 25.09.2026).

19. CREATE TRIGGER – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/lang_createtrigger.html (дата обращения: 25.09.2026).

20. Transaction – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/lang_transaction.html (дата обращения: 25.09.2026).

21. SQLite Foreign Key Support – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/foreignkeys.html (дата обращения: 25.09.2026).

22. SQL Features That SQLite Does Not Implement – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/omitted.html (дата обращения: 25.09.2026).

ПРИЛОЖЕНИЕ А

Сценарий создания объектов базы данных

Ниже приведены команды разделов `Up` миграций PostgreSQL в порядке их применения. Пароли в миграции ролей являются учебными значениями.

Основные таблицы

```
create table if not exists image (
    id serial primary key,
    name text not null,
    data bytea not null
);

create table if not exists location (
    id serial primary key,
    name text not null,
    longitude float not null,
    latitude float not null,
    address text not null
);

create table if not exists users (
    id serial primary key,
    name text not null,
    age int not null check (age > 0),
    login text not null,
    password text not null,
    created_at timestamp not null,
    city text not null,
    telegram_login text,
    avatar_id int references image(id)
);

create table if not exists event (
    id serial primary key,
    name text not null,
    description text,
    created_at timestamp not null,
    updated_at timestamp not null,
    event_date timestamp not null,
    telegram_chat_url text,
    city text not null,
    creator_id int not null references users(id) on delete cascade,
    location_id int not null references location(id),
    image_id int references image(id),
    deleted boolean not null default false
);

create table if not exists member (
    id serial primary key,
    user_id int not null references users(id) on delete cascade,
    event_id int not null references event(id) on delete cascade,
    role text not null
);

create table if not exists category (
    id serial primary key,
```

```
    subcategory_id int references category(id),
    name text not null constraint category_name_unique unique
);

create table if not exists event_category (
    event_id int not null references event(id) on delete cascade,
    category_id int not null references category(id),
    primary key (event_id, category_id)
);

create table if not exists preset (
    id serial primary key,
    name text not null,
    category_id int not null references category(id),
    creator_id int not null references users(id) on delete cascade
);

create table if not exists subscribe (
    id serial primary key,
    subscriber_id int not null references users(id) on delete cascade,
    user_id int not null references users(id) on delete cascade,
    subscribe_at timestamp not null
);
```

Ограничение логина и роль пользователя

```
alter table users
    add constraint users_login_unique unique (login),
    add column role text not null default 'user';
```

Категории и уникальность участия

```
create unique index if not exists member_user_event_unique
    on member (user_id, event_id);

insert into category (name)
values
    ('спорт'),
    ('музыка'),
    ('игры'),
    ('арт'),
    ('прогулки'),
    ('учёба'),
    ('еда'),
    ('tech'),
    ('путешествия'),
    ('фото'),
    ('кино')
on conflict (name) do nothing;
```

Отзывы

```
create table if not exists review (
    id bigserial primary key,
    event_id bigint not null
        references event(id) on delete cascade,
    author_id bigint not null
        references users(id) on delete cascade,
    rating integer not null
        constraint review_rating_check check (rating between 1 and 5),
    text text not null
        constraint review_text_length_check check (char_length(text) <= 2000),
```

```
    created_at timestamp not null default (now() at time zone 'utc'),
    constraint review_event_author_unique unique (event_id, author_id)
);

create index if not exists review_event_id_idx on review (event_id);
create index if not exists review_event_creator_lookup_idx on review (event_id);
```

Журнал и триггер удаления

```
create table if not exists event_deletion_log (
    id bigserial primary key,
    event_id int not null references event(id) on delete cascade,
    deleted_at timestamp not null default (now() at time zone 'utc')
);

-- +goose StatementBegin
create or replace function log_event_deletion()
returns trigger
language plpgsql
as $$
begin
    if old.deleted = false and new.deleted = true then
        insert into event_deletion_log (event_id)
        values (new.id);
    end if;

    return new;
end;
$$;
-- +goose StatementEnd

create trigger event_deletion_log_trigger
after update of deleted on event
for each row
execute function log_event_deletion();
```

Индекс поиска

```
create index if not exists event_search_city_date_idx
    on event (city, event_date, id)
    where deleted = false;
```

Роли и права доступа

```
-- +goose StatementBegin
do $$
begin
    if not exists (select 1 from pg_roles where rolname = 'joinup_guest') then
        create role joinup_guest login password 'guest';
    end if;
    if not exists (select 1 from pg_roles where rolname = 'joinup_user') then
        create role joinup_user login password 'user';
    end if;
    if not exists (select 1 from pg_roles where rolname = 'joinup_admin') then
        create role joinup_admin login password 'admin';
    end if;
    execute format('grant connect on database %I to joinup_guest, joinup_user, joinup_admin',
        current_database());
end;
$$;
-- +goose StatementEnd
```

```
grant usage on schema public to joinup_guest, joinup_user, joinup_admin;

revoke all privileges on all tables in schema public from joinup_guest, joinup_user, joinup_admin;
revoke all privileges on all sequences in schema public from joinup_guest, joinup_user, joinup_admin;

grant select on event, location, image, category, event_category, review to joinup_guest;

grant select, insert, update, delete on
    image,
    location,
    users,
    event,
    member,
    category,
    event_category,
    preset,
    subscribe,
    review,
    event_deletion_log
to joinup_user;

grant usage, select on
    image_id_seq,
    location_id_seq,
    users_id_seq,
    event_id_seq,
    member_id_seq,
    category_id_seq,
    preset_id_seq,
    subscribe_id_seq,
    review_id_seq,
    event_deletion_log_id_seq
to joinup_user;

grant all privileges on
    image,
    location,
    users,
    event,
    member,
    category,
    event_category,
    preset,
    subscribe,
    review,
    event_deletion_log
to joinup_admin;

grant all privileges on
    image_id_seq,
    location_id_seq,
    users_id_seq,
    event_id_seq,
    member_id_seq,
    category_id_seq,
    preset_id_seq,
    subscribe_id_seq,
    review_id_seq,
    event_deletion_log_id_seq
to joinup_admin;
```